COLLEGE NOTICE MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

NARENDARAN K

240701624

in partial fulfilment of the End Semester Examination for the course

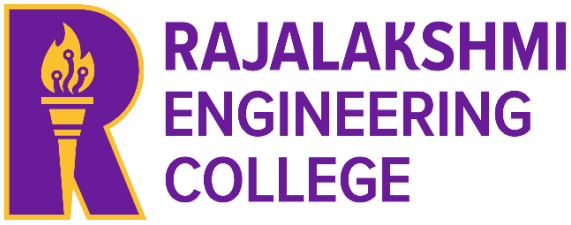
CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF ENGINEERING

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "COLLEGE NOTICE MANAGEMENT SYSTEM " is the bonafide work of NARENDARAN K (240701624), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

MANORANJINI
HEAD OF THE DEPARTMENT
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Dr. J. JINU SOPHIA
COURSE FACULTY
Department of Computer Science and Engineering
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on 11 November 2026.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, NARENDARAN K (240701624), hereby declare that the project report titled College management system",submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original work carried out by me individually. The Zoho Creator application described in this report — including its data model, forms, reports, pages, Deluge scripts, workflows and integrations — was designed and developed by me.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. Every template, sample application, AI-assisted feature (such as Zia or the AI application builder), code snippet or external resource used in this work has been duly acknowledged in the report.

Place: Chennai **SIGNATURE OF THE STUDENT**

NARENDARAN K

240701263

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman Thiru. S. Meganathan, B.E., F.I.E., our Chairperson Dr. (Mrs.) Thangam Meganathan and our Vice-Chairman Thiru. Abhay Shankar Meganathan, B.E., M.S. for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr. S. N. Murugesan, M.E., Ph.D., and to Dr. C. R. Muthukrishnan, Advisor, Rajalakshmi Institutions, for the constant encouragement extended to students to take up industry-relevant learning.

I sincerely thank Manoranjini, Head of the Department of Computer Science and Engineering, for the support and departmental facilities provided throughout this project.

I owe my sincere thanks to my course faculty, Aswana Lal, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

NARENDARAN K

ABSTRACT

Colleges share examination timetables, fee reminders, event announcements and academic circulars with students, faculty, non-teaching staff and Heads of Department. In the existing process these notices are typed, reviewed informally and then posted on physical notice boards, email threads and several messaging groups. As a result there is no single authoritative place to find current notices, no recorded approval trail, and no reliable way to confirm that each intended recipient has read or acknowledged an important message.

The College Notice Management System (CNMS) is a role-based application built on the Zoho Creator low-code platform to solve this problem. Its core modules are the Notice, Notice Recipient, Notice Approval Log, User Profile and Department forms, supported by Notice Category, Course and Notice Query. Deluge scripts validate every submission, route faculty notices to the Head of Department for approval, resolve the target audience by department, course, year and role, and create one recipient record per eligible user. Scheduled workflows publish approved notices on their publication date, send email notifications and reminders, and expire and archive old notices. Role-based access gives Admin, HOD/Principal, Faculty and Student/Staff users only the menus, records and actions they need, and dashboards summarise notice and acknowledgement activity.

The application was tested against functional and negative test cases covering validation, approval and rejection, audience targeting, publishing, read tracking and acknowledgement, and was published to the production environment with record-level permissions, attachment restrictions and an auditable approval log. CNMS gives the college one searchable source of notices, accountable approvals and recipient-level read and acknowledgement evidence, while demonstrating how low-code development can deliver a secure institutional workflow application quickly.

Keywords: College Notice Management System; Zoho Creator; Deluge; Workflow Automation; Notice Approval; Audience Targeting; Read Tracking; Role-Based Access Control

PROJECT AT A GLANCE

Attribute	Details
Project Title	College Notice Management System (CNMS)
Application Domain	Education – College communication and administration
Student Name & Register No.	NARENDARAN K– 240701624
Institution	Rajalakshmi Engineering College, Thandalam, Tamil Nadu
Course Code and Title	CB23F35 – Low-Code Application Development using Zoho Creator
Platform	Zoho Creator – web application with mobile access objective
Primary Users	Admin, Principal, HOD, Faculty, Student, Non-Teaching Staff
Core Modules	Notices, Notice Recipients, Notice Approval Log, User Profiles, Departments
Additional Data Entities	Notice Category, Course, Notice Query (confirm/create if absent)
Visible Reports	All Notices, Notice Types, All User Profiles, Approval Log Report, All Notice Recipients
Visible Dashboard	Summary cards, read-status counts, quick links, role and approval charts
Key Automation Required	Submission, approval/rejection, publish, notify audience, read/acknowledge, reminders, expiry, digest
Evidence Supplied	11 screenshots of the current Zoho Creator application
Academic Year	2026–2027
Report Date	09 November 2026
Implementation Status	Core forms/reports/dashboard visible; automation and permissions require verification

TABLE OF CONTENTS

ABSTRACT iv
PROJECT AT A GLANCE v
LIST OF TABLES ix
LIST OF FIGURES x
LIST OF ABBREVIATIONS xi
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 1
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 2
1.6 Organisation of the Report 2
LITERATURE SURVEY AND SYSTEM STUDY 3
2.1 Study of the Existing System 3
2.2 Limitations of the Existing System 3
2.3 Low-Code versus Traditional Development 4
2.4 Literature Survey 4
2.5 Emerging Trends in Low-Code Development 5
2.6 Proposed System 5
REQUIREMENT ANALYSIS 6
3.1 Stakeholders and User Roles 6
3.2 Functional Requirements 6
3.3 Non-Functional Requirements 6
3.4 Hardware and Software Requirements 7
3.5 Use Case Model 8
SYSTEM DESIGN 9
4.1 System Architecture 9
4.2 Data Model 10
4.3 Form Design 10
4.4 Process and Workflow Design 11
4.5 User Interface and Navigation Design 12
4.6 Access Control Design 12
IMPLEMENTATION 13
5.1 Application Setup 13
5.2 Forms and Data Capture 13

5.3 Reports 15
5.4 Pages and Dashboards 16
5.5 Deluge Scripting 16
5.6 Workflows and Automation 17
5.7 Integrations 18
5.8 User Management and Role-Based Access 19
5.9 AI and Analytics Features 20
5.10 Pro-Code Extension 21
5.11 Mobile and Portal Access 22
TESTING AND DEBUGGING 23
6.1 Testing Strategy 23
6.2 Test Cases 23
6.3 Debugging Deluge Scripts 24
6.4 Defect Log 25
6.5 User Acceptance Testing 25
DEPLOYMENT, SECURITY AND GOVERNANCE 26
7.1 Environments and Deployment Strategy 26
7.2 Performance Optimisation 27
7.3 Security Features 27
7.4 Governance and Administration 27
RESULTS AND DISCUSSION 29
8.1 Application Output 29
8.2 Achievement of Objectives 30
8.3 Comparison with the Existing System 31
8.4 Limitations of the Low-Code Approach 31
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 31
CONCLUSION AND FUTURE ENHANCEMENTS 32
9.1 Conclusion 32
9.2 Future Enhancements 32
9.3 Learning Outcomes and Career Relevance 32
REFERENCES 33
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 34
APPENDIX B – COURSE OUTCOME MAPPING 36
APPENDIX C – COMPLETE DELUGE SOURCE CODE 37
APPENDIX D – USER MANUAL 42
D.1 Accessing the Application 42
D.2 Instructions for [Role 1] 42

D.3 Instructions for [Role 2]..... 42
APPENDIX E – EVALUATION RUBRIC..... 43
APPENDIX F – VIVA-VOCE RECORD SHEET..... 44

LIST OF TABLES

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application 4
Table 2.2: Summary of Literature Survey 4
Table 3.1: Stakeholders and User Roles 6
Table 3.2: Functional Requirements 6
Table 3.3: Non-Functional Requirements 6
Table 3.4: Hardware and Software Requirements 7
Table 3.5: Use Case Description – [Use Case Name] 8
Table 4.1: Entities and Relationships 10
Table 4.2: Field Specification – [Form Name] Form 11
Table 4.3: Role–Permission Matrix 12
Table 5.1: Reports Implemented 15
Table 5.2: Deluge Scripts Implemented 16
Table 5.3: Workflows Implemented 17
Table 5.4: Integrations Implemented 18
Table 6.1: Test Cases 23
Table 6.2: Defect Log 25
Table 6.3: User Acceptance Feedback 25
Table 7.1: Deployment Checklist 26
Table 7.2: Performance Optimisation Results 27
Table 7.3: Security Controls Implemented 27
Table 8.1: Mapping of Objectives to Outcomes 30
Table 8.2: Existing System versus Proposed Application 31
Table A.1: Feature Coverage Checklist 34
Table B.1: Mapping of Course Outcomes to Report Chapters 36
Table E.1: Evaluation Rubric for the Individual Project 43
Table E.2: Performance Level Descriptors 43
Table F.1: Viva-Voce Questions and Responses 44
Table F.2: Marks Summary 44

LIST OF FIGURES

Figure 2.1: [As-Is Process Flow of the Existing System] 3
Figure 3.1: [Use Case Diagram of the Application] 8
Figure 4.1: [System Architecture of the Application] 9
Figure 4.2: [Entity–Relationship Diagram] 10
Figure 4.3: Workflow / Process Flow for [Process Name] 11
Figure 4.4: Navigation Structure and Wireframe of [Page Name] 12
Figure 5.1: [Application Home and Navigation Sections] 13
Figure 5.2: CNMS Notice Form in Zoho Creator 14
Figure 5.3: [Form Name] Form in Live Mode 14
Figure 5.4: [Report Name] Report 15
Figure 5.5: [Dashboard Page Name] 16
Figure 5.6: [Workflow / Blueprint] Configuration 18
Figure 5.7: [Connection Setup and Postman / Response Output] 19
Figure 5.8: User Profiles and Role Configuration Screen 20
Figure 5.9: [AI / Analytics Feature in Use] 21
Figure 5.10: [Application on Mobile / Customer Portal] 22
Figure 6.1: [Deluge Debug Output / Log] 24
Figure 7.1: [Environments / Publish to Production] 26
Figure 7.2: [Audit Trail / Administration Settings] 28
Figure 8.1: All Notices Report – Published Notice Records 29
Figure 8.2: Notice Recipient Records and Read/Acknowledgement Tracking 30
Figure 8.3: Notice Approval Log Report 30

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
CNMS	College Notice Management System
CRUD	Create, Read, Update, Delete
ER	Entity–Relationship
HOD	Head of Department
LCAP	Low-Code Application Platform
RBAC	Role-Based Access Control
UAT	User Acceptance Testing
UI	User Interface
PDF	Portable Document Format
JPG	Joint Photographic Experts Group
PNG	Portable Network Graphics
ID	Identifier
FR	Functional Requirement

CHAPTER 1
INTRODUCTION

1.1 Background

Colleges distribute examination timetables, fee reminders, laboratory safety instructions, event announcements, library-hour changes and academic circulars to several groups. These groups include students, faculty, non-teaching staff, Heads of Department and administrators. The intended audience differs by notice: a circular may apply to everyone, while an examination update may apply only to students in a particular course and year. CNMS provides a central application in which each notice is stored with its category, priority, schedule, audience and lifecycle status. The system is designed to reduce dependence on physical notice boards and fragmented messaging groups.

1.2 Problem Statement

The existing communication process does not consistently provide one authoritative location for current notices, a controlled approval trail, precise audience distribution or evidence that each intended recipient has read and acknowledged important information. As a result, important messages may be missed, outdated copies may remain in circulation, and administrators may be unable to produce a reliable read/acknowledgement report.

- Notices can be missed or buried in multiple WhatsApp groups, email threads and paper notice boards.
- Without a defined approval workflow, it may be unclear who authorised a notice and whether its content was reviewed.
- The institution may lack recipient-level proof of reading, acknowledgement, expiry handling and a searchable historical record.

1.3 Objectives

- To design structured data forms for notices, recipients, approval history, departments, users, categories, courses and queries.
- To specify and implement workflows for submission, approval/rejection, publishing, audience notification, read tracking, acknowledgement, queries and scheduled reminders.

- To provide role-specific dashboards, reports and access rules for administrators, approvers, publishers and recipients.
- To use Zoho Creator email capabilities and supported connections to notify approvers, publishers and targeted recipients.
- To document secure deployment, validation, audit, retention and governance controls for the college application.

1.4 Scope of the Project

In scope: notice creation and editing; approval and rejection; audience targeting; publishing; email notification; recipient/read/acknowledgement tracking; notice queries; expiry and archival; dashboards; reports; and role-based access. Out of scope: processing fee payments, replacing the student information system, changing the official academic calendar, and guaranteeing external email delivery. The platform is Zoho Creator, accessed through a web browser and intended to be mobile-friendly.

1.5 Justification for a Low-Code Approach

Zoho Creator is suitable because CNMS is primarily a data-centric workflow application. Forms and lookup relationships represent notices, users, departments and recipient activity. Reports and pages provide searchable lists and role-specific dashboards. Deluge can implement validation, audience resolution and state transitions. Built-in hosting and mobile access reduce infrastructure work, while permissions and audit facilities support governance. Low-code does not remove the need for careful data modelling, security configuration and testing [2], [3].

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related low-code literature. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

CHAPTER 2
LITERATURE SURVEY AND SYSTEM STUDY

2.1 Study of the Existing System

The existing process commonly begins when a staff member prepares a circular. The content is shared with a supervisor for informal review, then distributed through one or more channels such as a notice board, email or messaging group. Recipients must locate the message themselves. When a notice changes, the updated version may not reach every group. There is no consistent recipient list linked to the notice, and the college may need to manually collect acknowledgement evidence. CNMS formalises these steps through a notice record, approval record and recipient record.

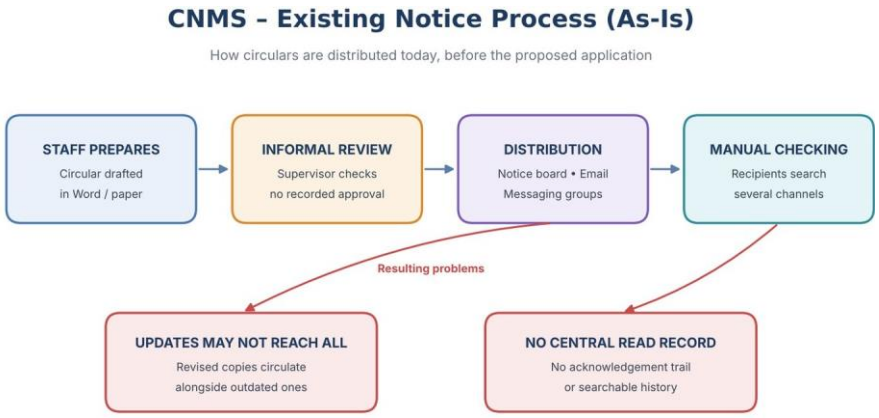


Figure 2.1: [As-Is Process Flow of the Existing System]

2.2 Limitations of the Existing System

- Information is fragmented across channels and may be duplicated or overlooked.
- Approval responsibility and rejection reasons are not consistently recorded in one auditable log.
- Read status, acknowledgement, reminder and expiry information may require manual follow-up.

2.3 Low-Code versus Traditional Development

Table 2.1: Comparison of Low-Code and Traditional Development for the Proposed Application

Criterion	Traditional (Pro-Code) Development	Low-Code on Zoho Creator
Development time	Requires custom UI, database, authentication and workflow implementation; effort depends on team	Creator forms, reports and workflow builders accelerate the initial application build
Skills required	Frontend, backend, database, hosting and deployment skills	Creator configuration, data modelling and Deluge skills
Hosting & infrastructure	Separate hosting, monitoring, backup and deployment setup	Platform hosting and managed environment, subject to subscription
Workflow changes	Often require code changes and redeployment	Many changes can be made through visual configuration and Deluge
Maintenance	Team maintains application stack and infrastructure	College maintains Creator configuration, data, roles and platform subscription
CNMS suitability	Suitable for extensive custom integrations and interfaces	Suitable for structured notices, approvals, reports and scheduled automation
Customisation limits	[ ]	[ ]
Cost	[ ]	[ ]

2.4 Literature Survey

Table 2.2: Summary of Literature Survey

S.No.	Author(s) / Source & Year	Title / Product	Key Findings	Relevance to this Project
1	Zoho Corporation, current documentation	Zoho Creator Help	Documents forms, reports, pages, workflows and application administration.	Primary platform reference for CNMS implementation.
2	Zoho Corporation, current documentation	Deluge Script Reference	Describes Deluge syntax, tasks, collections and workflow scripting.	Supports validation, recipient resolution and notification logic.
3	Zoho Corporation, current documentation	Zoho Creator – Pages and Reports	Explains dashboard pages, report layouts and visualisation features.	Relevant to role-based dashboards and operational reports.
4	OWASP Foundation	Application Security Verification Standard	Provides verification areas for authentication, access control and data protection.	Informs role and record-level security test design.
5	Pressman and Maxim, 2019	Software Engineering: A Practitioner's Approach	Explains requirements, design, testing and maintenance practices.	Provides the software engineering basis for the project lifecycle.

2.5 Emerging Trends in Low-Code Development

Relevant low-code trends include visual application assembly, reusable workflow automation, AI-assisted development, integration through APIs and the use of analytics dashboards. For CNMS, visual forms and reports support rapid iteration with college staff. Workflow automation reduces repetitive routing and reminder tasks. Analytics can identify notices with low acknowledgement. AI-assisted development may help draft or explain scripts, but generated logic must be reviewed and tested before use. Sensitive student data should not be sent to unapproved AI services.

2.6 Proposed System

The proposed CNMS stores notices centrally and connects each notice to its creator, department, approver and recipient list. Faculty submit notices for review. HODs or Principals approve or reject them with comments. Approved notices are published when their date arrives and distributed to users matching the audience criteria. The application records read and acknowledgement events and provides reports for follow-up. Admin maintains users, departments and configuration.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Admin	Application administrator with full access.	Manage users, departments, categories, courses, settings, all notices and reports.
Principal / HOD	Authorised approver for the relevant department or institution.	Review pending notices; approve or reject with remarks; monitor department notices.
Faculty	Notice publisher and creator.	Create and edit own Draft/Rejected notices; submit for approval; monitor read and acknowledgement.
Student / Non-Teaching Staff	Notice recipients with role-specific visibility.	View published notices targeted to them; search, read, acknowledge and ask questions where enabled.

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall capture notice title, category, description, audience and publication date as mandatory fields.	Admin, HOD, Faculty	High
FR-02	The system shall validate expiry after publication, restrict past dates for non-admin users and prevent duplicate title/category/date entries.	All publishers	High
FR-03	The system shall validate attachments as PDF/JPG/PNG and enforce a maximum size of 10 MB.	All publishers	High
FR-04	The system shall route faculty submissions to the HOD of the creator's department and notify the approver.	Faculty, HOD	High
FR-05	The system shall approve or reject notices, require a rejection reason and append every decision to the Approval Log.	HOD, Principal, Admin	High
FR-06	The system shall publish approved notices when the publication date is reached and resolve the configured audience.	System, Admin	High
FR-07	The system shall create one recipient row per eligible active user and prevent duplicate notice/user rows.	System	High

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	Common reports should load within an institutionally agreed response target under the expected record volume; measure during UAT.

NFR ID	Category	Requirement
NFR-02	Security	Users shall access only the records and actions permitted by their role and department.
NFR-03	Usability	The interface shall be understandable to a new user and usable on desktop and supported mobile devices.
NFR-04	Reliability	Scheduled workflows shall be safe to rerun without creating duplicate recipient rows.
NFR-05	Maintainability	Forms, field link names, workflows and functions shall use consistent naming and documentation.
NFR-06	Auditability	Approval decisions, read events and acknowledgements shall retain timestamps and related user identities.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator (edition/licence to be confirmed by institution)
Scripting language	Deluge
Browser	Google Chrome / Microsoft Edge / Mozilla Firefox current supported versions
Client devices	Desktop/laptop and supported Android/iOS mobile devices
Connectivity	Internet connection required for hosted application and email notifications
Data storage	Zoho Creator application data and attached files, subject to plan limits
Integration	Authorised email connection; Excel/PDF export features as configured

3.5 Use Case Model

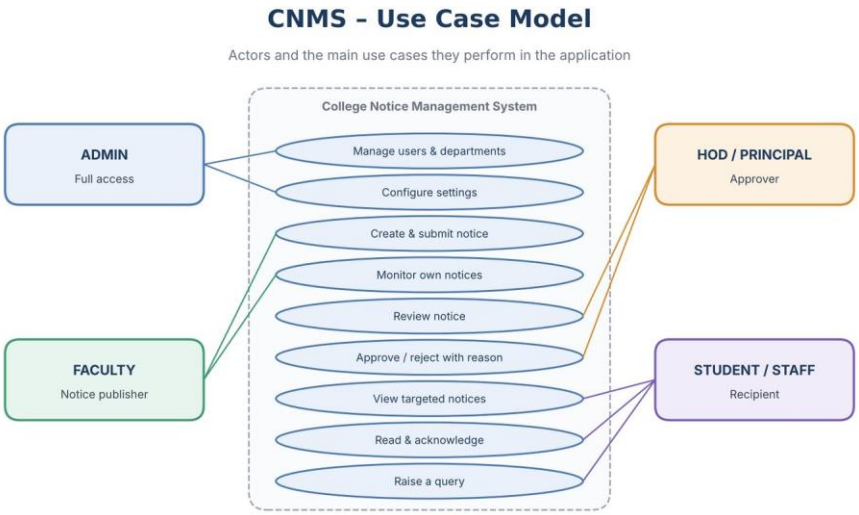


Figure 3.1: [Use Case Diagram of the Application]

Table 3.5: Use Case Description – [Use Case Name]

Use Case ID	UC-01
Use Case Name	Create and submit a notice for approval
Actor(s)	Faculty publisher
Pre-condition	User is signed in, active and assigned to a department; required category and audience values exist.
Main Flow	Open Notice form; enter title/content/category/priority; choose audience; set dates; attach file if required; review and submit.
Post-condition	Notice is saved as Pending Approval and the relevant HOD is notified.
Alternative Flow	If validation fails, submission is blocked and the user receives a field-level message.
Exception	If the department has no active HOD mapping, submission must be flagged for Admin correction.

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

The architecture has four logical layers. The presentation layer contains Creator forms, reports, pages and mobile views for each role. The application layer contains validation rules, Deluge functions, approval actions and scheduled workflows. The data layer stores Notice, Notice Recipient, Notice Approval Log, User Profile, Department and supporting Category, Course and Query records. The integration layer handles email notifications and authorised exports or external connections. A user action is checked against role and record scope before it changes data.

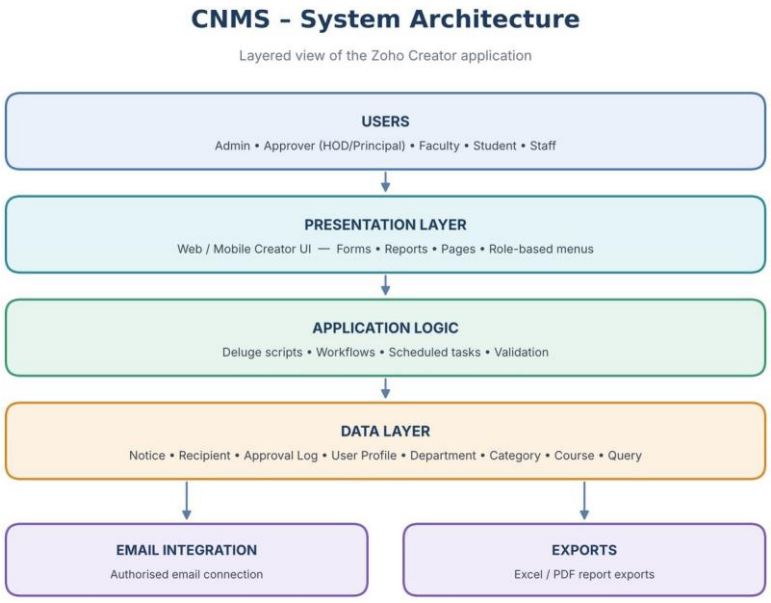


Figure 4.1: [System Architecture of the Application]

A user signs in through the Creator interface. The application presents only the modules and records allowed for that role. Form submissions invoke validation and workflow logic. Successful actions update the related data records and create audit entries. Email notifications are sent after the relevant state transition. Reports read the stored records and aggregate them into operational indicators. The design should use authenticated record links and avoid exposing recipient details through public URLs.

4.2 Data Model

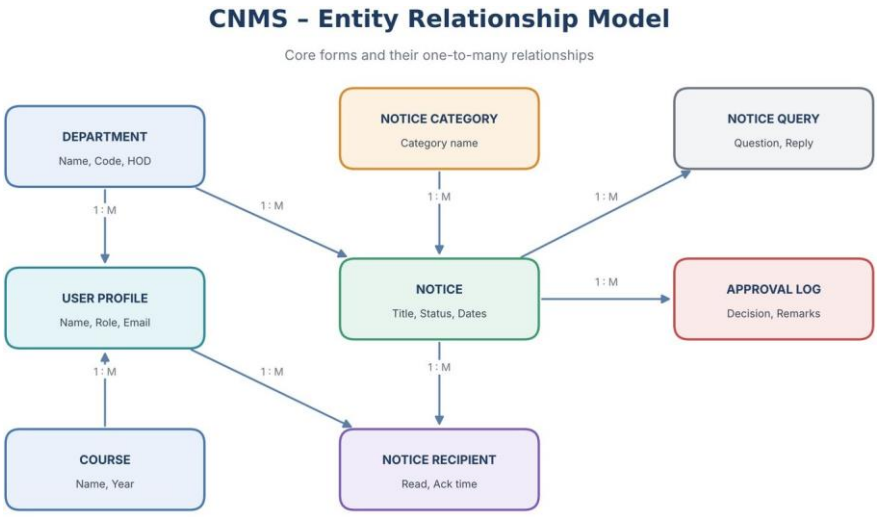


Figure 4.2: [Entity–Relationship Diagram]

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Notice	Notice ID, title, category, content, priority, dates, audience, status	Notice Recipient; Approval Log; Notice Query	One-to-Many	Lookup in child forms
Notice Recipient	Notice, User, Read, Read Time, Acknowledged, Acknowledged Time	Notice; User Profile	Many recipient rows to one notice/user	Lookup fields
Notice Approval Log	Notice, approver, role, decision, remarks, date-time	Notice; User Profile	Many log rows to one notice	Lookup fields

4.3 Form Design

The core forms are Notice, Notice Approval Log, Notice Recipient, Department and User Profile. Do not duplicate these existing forms. Add missing fields to the existing forms and create Notice Category, Course and Notice Query only if they do not already exist. The Notice form requires a unique ID, title, category, description, attachment, priority, dates, pin flag, acknowledgement flag, audience selectors, creator, approver, rejection reason and status. Recipient records require notice/user lookups plus read and acknowledgement flags and timestamps. User Profile requires role, department, course, year, ID number and active status.

Table 4.2: Field Specification – [Form Name] Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Notice ID	Notice_ID	Auto Number	System	Unique	Generated by Creator
Notice Title	Notice_Title	Single Line	Yes	Not blank; duplicate rule by category/date	Visible in All Notices
Notice Content	Notice_Content	Multi Line	Yes	Not blank	Description/ body
Notice Category	Notice_Category	Lookup	Yes	Must reference active category	Add Category form if absent
Priority	Priority	Drop Down	Yes	Normal / Important / Urgent	Standardise current values

4.4 Process and Workflow Design

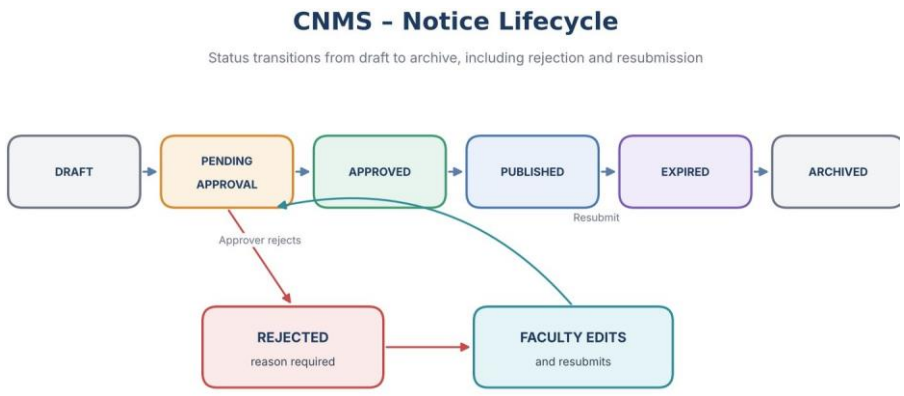


Figure 4.3: Workflow / Process Flow for [Process Name]

4.5 User Interface and Navigation Design

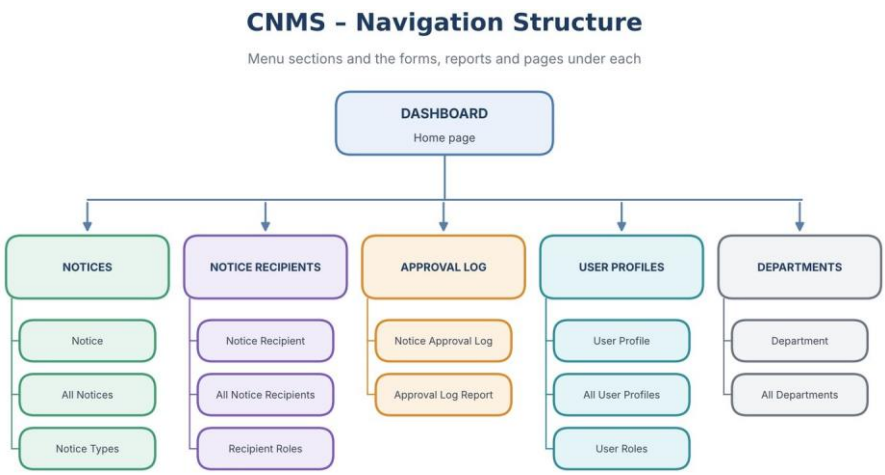


Figure 4.4: Navigation Structure and Wireframe of [Page Name]

4.6 Access Control Design

Table 4.3: Role–Permission Matrix

Component	Admin	Principal/HOD	Faculty	Student/Staff
Notice form	A E V D	V/A approval in scope	A E own Draft/Rejected V own	V Published targeted
Notice Recipient	A E V D	V department	V own notice recipients	V own; update read/ack only
Approval Log	A E V D	A V in scope	V own notices	—
User Profile	A E V D	V department if required	V limited directory if permitted	V own profile only
Department / Category / Course	A E V D	V	—	—

CHAPTER 5
IMPLEMENTATION

The current application is titled “College Notice Management System”. The supplied screenshots show a Creator workspace with a left navigation panel. The visible menu contains Dashboard, Notice Recipients, Notice Approval Log, User Profiles, Departments and Notices. The Notices menu expands to Notice, All Notices and Notice Types. The application uses an orange accent in the captured screens, while the project design requirement specifies a blue-and-white theme with red for Urgent and green for Approved. The final theme should be made consistent before release.

5.1 Application Setup

The CNMS design and evidence for this section are described in the surrounding text, tables and supplied screenshots. Any remaining configuration must be verified in the running Zoho Creator application before final submission.

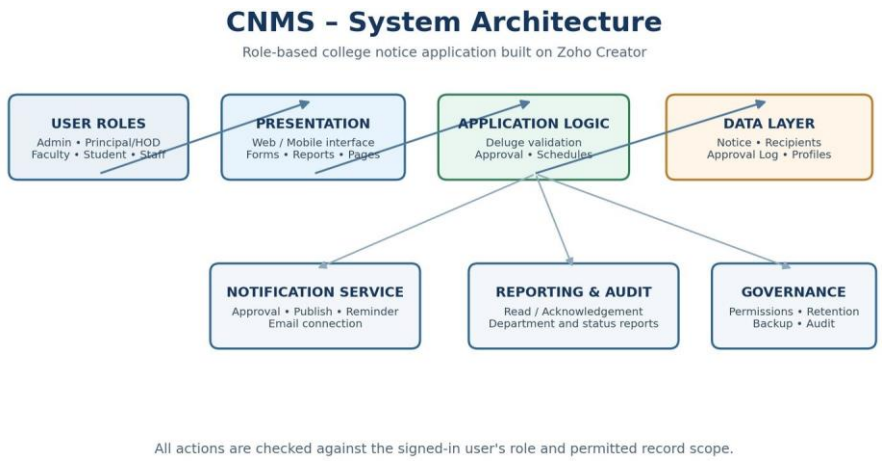


Figure 5.1: [Application Home and Navigation Sections]

5.2 Forms and Data Capture

The Notice form currently displays Notice Title, Notice Content, Notice Type, Priority, Target Audience, Publication Date, Expiry Date, Created By, Approval Status and Is Urgent. The Department form displays Department Name, Department Code, Head of Department, Contact Email, Contact Phone, Location, Active Status and Created Date. User Profiles displays user name, email, phone, role, department, designation, active

status and created date. The Approval Log and Notice Recipient forms are also visible. Add the additional required fields without creating duplicate forms.

The CNMS design and evidence for this section are described in the surrounding text, tables and supplied screenshots. Any remaining configuration must be verified in the running Zoho Creator application before final submission.

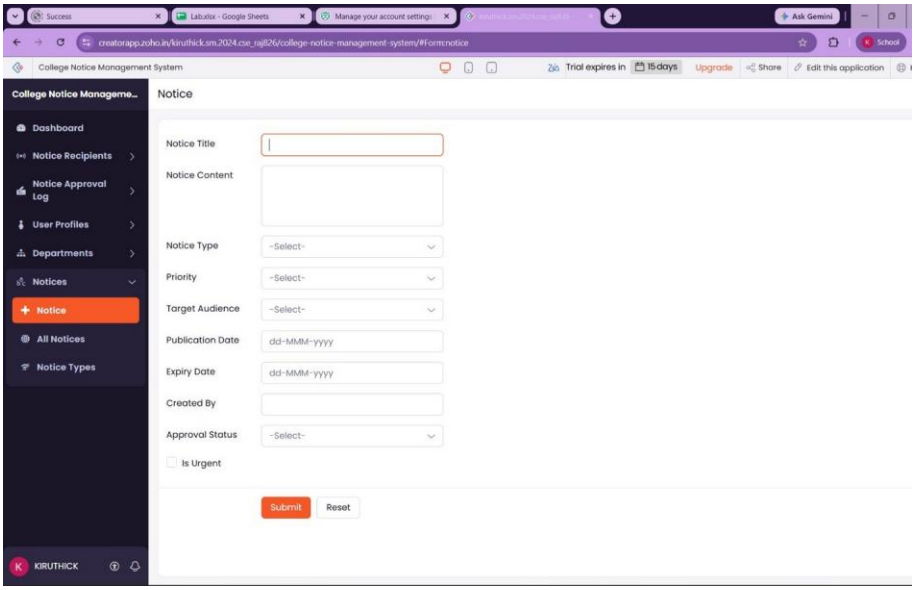


Figure 5.2: CNMS Notice Form in Zoho Creator

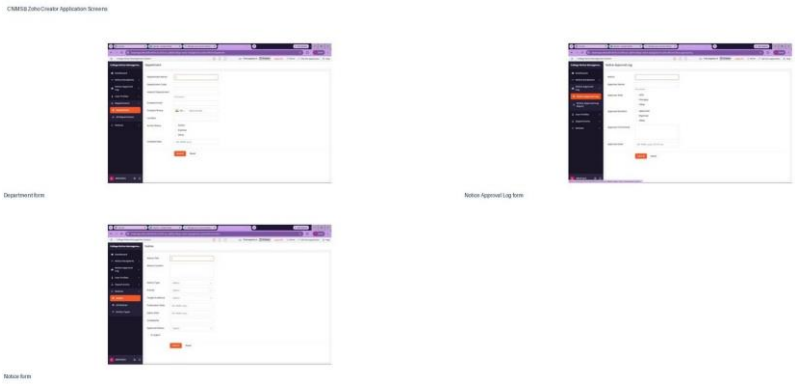


Figure 5.3: [Form Name] Form in Live Mode

5.3 Reports

The screenshots show All Notices as a tabular report with title, content, type, priority, target audience, publication date and expiry date. Notice Types groups notices under Urgent, General, Event and Academic. Other visible reports include All User Profiles, Notice Approval Log Report and All Notice Recipients. The final reporting set should also include read/unread per notice, acknowledgement export, department-wise notices and expired/archived notices. Apply role-specific criteria to every report.

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
All Notices	List	Notice	Filter by status, category, priority and date; sort by publish date	Open / Edit within permission	Admin, HOD, Faculty
Notice Types	Grouped report	Notice	Group by category/type	Open notice	Admin, Faculty
All User Profiles	List	User Profile	Filter active status, role and department	Admin maintenance	Admin
Notice Approval Log Report	List	Notice Approval Log	Sort by approval date descending	Open linked notice	Admin, HOD/Principal
All Notice Recipients	List	Notice Recipient	Filter by notice, role, read and acknowledgement	Open recipient record	Admin, authorised publisher

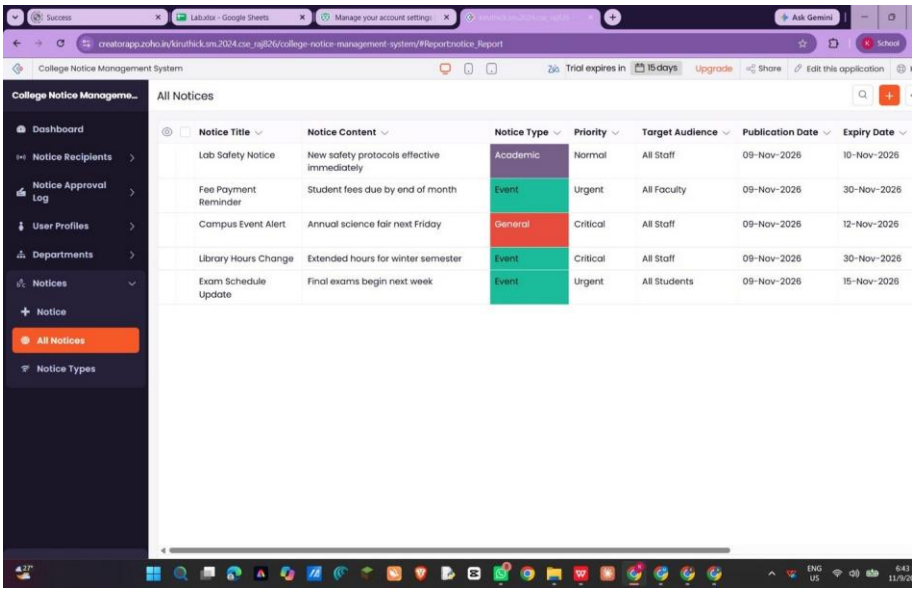


Figure 5.4: [Report Name] Report

5.4 Pages and Dashboards

The dashboard screenshot shows four summary cards, read-status counts, quick links for adding recipients, approval logs, user profiles and departments, and two charts: distribution by recipient role and approval decision. The final design should provide separate dashboard views for Student/Staff, Faculty, HOD/Principal and Admin. Student/Staff pages prioritise pinned and urgent notices, search, filters and pending acknowledgements. Faculty pages show personal notices by status and read percentage. Approver pages show pending approvals. Admin pages show category, department and monthly statistics.

The CNMS design and evidence for this section are described in the surrounding text, tables and supplied screenshots. Any remaining configuration must be verified in the running Zoho Creator application before final submission.

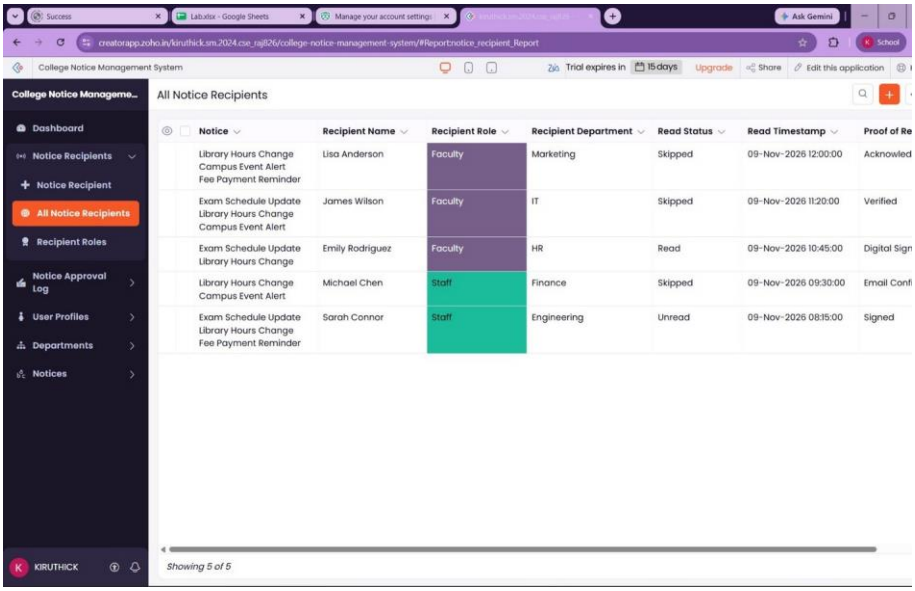


Figure 5.5: [Dashboard Page Name]

5.5 Deluge Scripting

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Notice → On Validate	Validate required fields, dates, duplicates and attachment constraints.	if/else, criteria fetch, alert, cancel submit
DS-02	Notice → On Success / Submit action	Set Pending Approval and notify department HOD; auto-approval exception for Admin/HOD.	lookup, if/else, sendmail

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-03	Function: Notify_Audience(Notice _ID)	Resolve active profiles and create recipient rows once per notice/user.	criteria fetch, for each, insert, sendmail
DS-04	Notice detail / recipient action	Record first read and separate acknowledgement timestamp.	lookup, conditional update, date-time
DS-05	Scheduled workflow	Publish due notices, send reminders, expire/archive notices and send weekly digest.	schedule, criteria, date arithmetic, function call

Code 5.1: [Validation script – replace with your own]

```
// DS-01 | Notice form | On Validate
// Check mandatory fields, expiry > publish date, past-date exception,
// duplicate title/category/date and allowed attachment extension/size.
// Use the actual field link names from the live Creator form.
// This is a design outline; paste and test the final Deluge script here.
```

Code 5.2: [Notification script – replace with your own]

```
// DS-02 | Notice submission workflow
// Resolve creator profile and department HOD.
// If creator role is Admin or HOD, apply the approved auto-approval policy.
// Otherwise set Pending Approval and notify the active HOD.
// Insert an Approval Log row for every approval/rejection action.
```

The intended Deluge logic includes server-side validation, duplicate detection, audience resolution, recipient deduplication, approval logging and notification dispatch. The exact Creator field link names and workflow triggers must be confirmed in the live builder before scripts are pasted. No script should be labelled as tested unless it has been executed successfully in the application. Full scripts belong in Appendix C after implementation.

5.6 Workflows and Automation

Table 5.3: Workflows Implemented

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Notice Validation	Form Workflow	On Validate	Required fields and business rules	Block invalid submission	DS-01
Notice Submission	Form Workflow	On Submit	Faculty creator	Set Pending Approval; email HOD	DS-02
Notice Approval	Approval / Action	Approver action	Authorised department approver	Approve or reject; require reason; append log	DS-02

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Daily Publish	Schedule	Daily 06:00	Approved and Publish Date reached	Publish and invoke Notify Audience	DS-03
Reminder / Expiry / Digest	Schedules	Daily 09:00 / 00:00; Monday 08:00	Age, acknowledgement and expiry conditions	Remind, alert, expire, archive, digest	DS-05

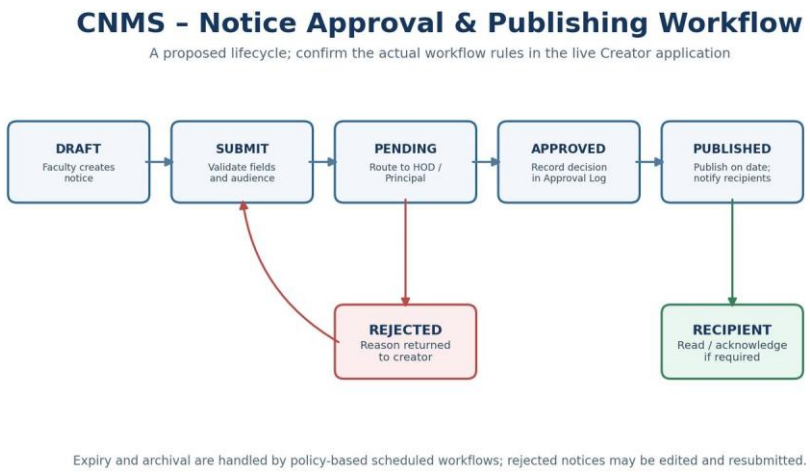


Figure 5.6: [Workflow / Blueprint] Configuration

5.7 Integrations

The primary integration is email notification through an authorised Zoho Creator connection or supported send-mail action. The application needs messages for HOD approval requests, rejection notices, publication notices, student queries and replies, acknowledgement reminders, expiry warnings and weekly digests. Email sender identity, connection permissions and recipient address validity must be checked. The supplied screenshots do not demonstrate successful email delivery.

Table 5.4: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
Email notifications	Zoho Creator email / authorised sender	Send mail task or configured connection	Authorised Zoho connection	Approval, rejection, publication, query, reminders and digest
Excel/PDF export	Creator report export	Built-in report export / configured action	Creator user permission	Acknowledgement and department reports

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
External identity sync (future)	College SIS / directory API	Creator REST API or webhook	OAuth 2.0 / approved token method	Synchronise users, courses and year

Code 5.3: [Integration script – replace with your own]

```
// Email integration design
// Use an authorised Creator connection or configured send-mail action.
// Never hard-code passwords or API secrets in Deluge source.
// Verify sender identity, recipient address, delivery status and error handling.
// The supplied screenshots do not prove that email delivery has been tested.
```

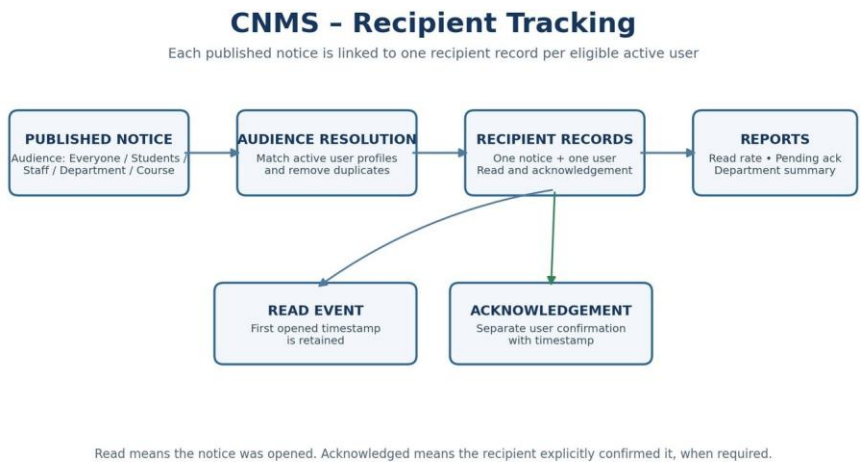


Figure 5.7: [Connection Setup and Postman / Response Output]

5.8 User Management and Role-Based Access

Role configuration must enforce Admin full access; HOD/Principal approval within their department; Faculty ownership of Draft and Rejected notices; and Student/Staff access only to Published notices targeted to them. Record-level criteria should use the authenticated user and linked profile rather than relying only on hidden menu items. Students should not see other users' recipient rows. Faculty should not edit another publisher's records. Approval actions must re-check the approver's department at execution time.

Configure application users with the roles Admin, Principal, HOD, Faculty, Student and Non-Teaching Staff. Map every user to an active User Profile. Use record criteria to restrict faculty to their own Draft/Rejected notices and own reports, approvers to notices in their department, and recipients to Published notices addressed to them. Keep the

recipient's read/acknowledgement action limited to their own recipient row. Admin has full access.

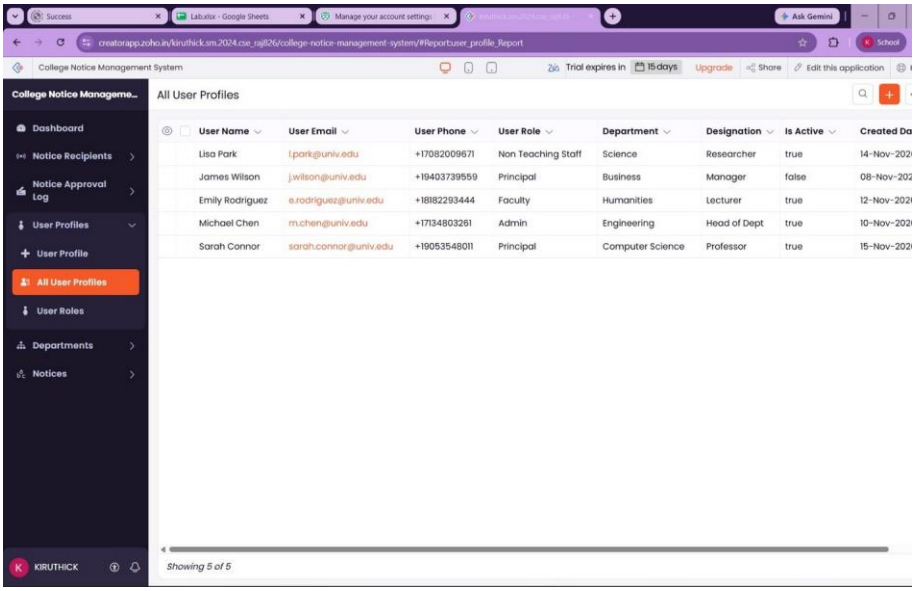


Figure 5.8: User Profiles and Role Configuration Screen

5.9 AI and Analytics Features

The dashboard already displays summary cards and charts, providing a basic analytics foundation. The report specification calls for counts by category, department and month, unread notice ranking and acknowledgement rates. No AI/Zia feature is evidenced in the supplied screenshots. AI-assisted generation may be used during development only if the resulting scripts are reviewed, tested and documented. Do not claim an AI feature is implemented unless it is visible and demonstrable in the live application.

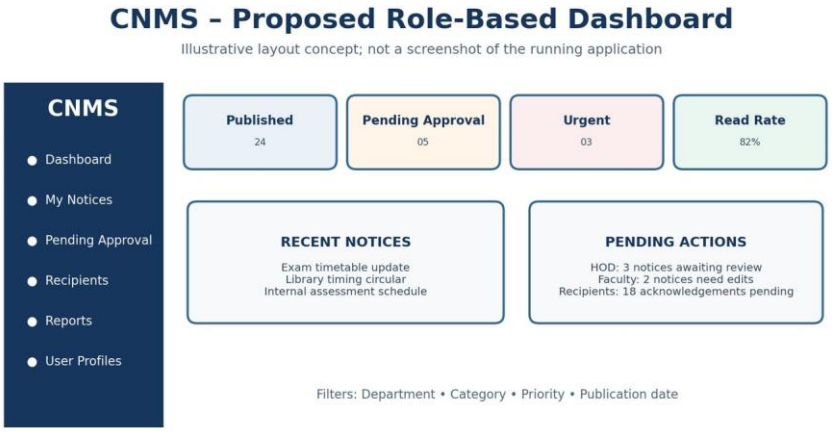


Figure 5.9: [AI / Analytics Feature in Use]

No separate pro-code component is visible in the supplied evidence. A future Creator REST API, JavaScript widget or Zoho Catalyst component may be considered if the college requires a custom interface or external processing not adequately supported by Creator. The current project can be implemented primarily with Creator configuration and Deluge.

No separate pro-code extension is evidenced in the supplied screenshots. The current solution is designed around Creator forms, reports, pages and Deluge. If future requirements exceed platform limits, a Creator REST API, JavaScript widget or Zoho Catalyst service could handle specialised processing. Such an extension is optional and should be justified by a concrete requirement.

5.11 Mobile and Portal Access

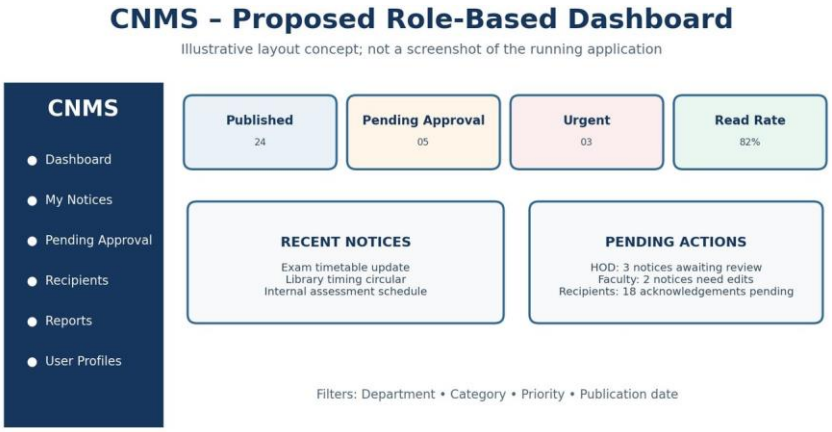


Figure 5.10: [Application on Mobile / Customer Portal]

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing should cover form validation, lookup integrity, status transitions, audience selection, recipient deduplication, notifications, schedules, exports and access restrictions. The supplied screenshots establish the presence of visible forms and reports, but they do not provide execution logs for background automation. Therefore, implementation claims should be separated from proposed test expectations. Use separate test accounts for Admin, HOD, Faculty, Student and Non-Teaching Staff.

6.2 Test Cases

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01	Submit notice with mandatory fields	Enter valid title/category /content/audience/date	Notice submitted	To be recorded during live test	Pending
TC-02	FR-01	Submit with title missing	Leave title blank	Validation error; no record submitted	To be recorded	Pending
TC-03	FR-02	Expiry before publication	Set expiry earlier than publish date	Submission blocked	To be recorded	Pending
TC-04	FR-02	Past date as Faculty	Choose yesterday	Submission blocked	To be recorded	Pending
TC-05	FR-02	Past date as Admin	Choose yesterday with Admin role	Allowed only if policy permits	To be recorded	Pending
TC-06	FR-03	Upload unsupported file	Attach DOCX	File rejected	To be recorded	Pending
TC-07	FR-03	Upload file over 10 MB	Attach oversized PDF	File rejected	To be recorded	Pending
TC-08	FR-04	Faculty submits notice	Use active department/ HOD mapping	Pending Approval and HOD notification	To be recorded	Pending
TC-09	FR-05	Approver approves notice	Use authorised HOD	Approved status and log row	To be recorded	Pending
TC-10	FR-05	Reject without reason	Select Reject; leave reason blank	Action blocked	To be recorded	Pending
TC-11	FR-05	Reject with reason	Enter rejection reason	Rejected; creator notified; log row	To be recorded	Pending
TC-12	FR-06	Publish due notice	Approved notice with date today	Published and target audience resolved	To be recorded	Pending

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-13	FR-07	Rerun publishing schedule	Run twice for same notice	No duplicate recipient rows	To be recorded	Pending
TC-14	FR-08	Open notice twice	Same recipient opens twice	First-read time retained	To be recorded	Pending
TC-15	FR-08	Acknowledge notice	Select I have read this	Acknowledgement and timestamp stored	To be recorded	Pending

6.3 Debugging Deluge Scripts

Debugging should use Deluge info statements, the editor's execute/debug facilities, workflow execution history and Creator error messages. For each defect, record the exact form, field link name, event, test record, user role and observed error. A common risk in CNMS is a missing lookup value or inactive HOD profile causing approval routing to fail. Another risk is duplicate recipient creation when a daily publishing schedule runs more than once. These cases should be deliberately tested and logged.

[Debugging approach.]

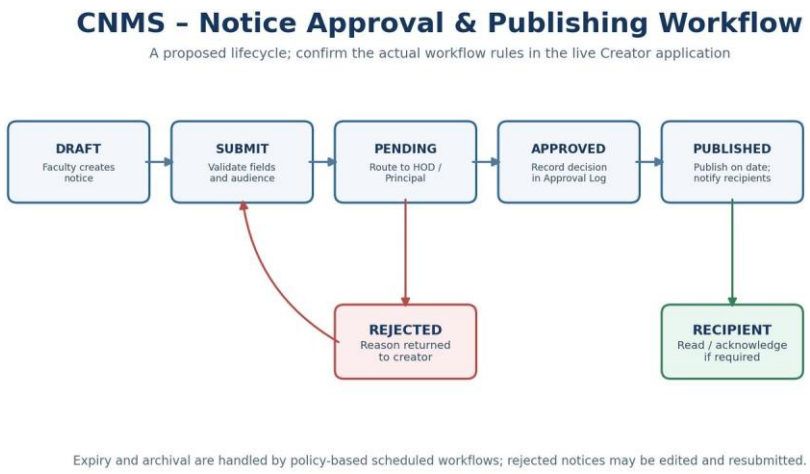


Figure 6.1: [Deluge Debug Output / Log]

6.4 Defect Log

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-01	Priority options differ from project requirement	Captured report shows Normal, Urgent and Critical; required values are Normal, Important and Urgent.	Standardise dropdown values and update existing sample records.	To verify
BUG-02	Student recipient group has no visible records	Recipient Roles screenshot shows No Records in Student.	Create student test profiles and target student notices; rerun recipient generation.	To verify

6.5 User Acceptance Testing

Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken
[Tester 1]	Faculty	[Record feedback after live demonstration]	[Record correction or confirm no change]
[Tester 2]	Student / Staff	[Record feedback after live demonstration]	[Record correction or confirm no change]

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

The deployment process should separate development changes from the live application. Before publishing, confirm form fields, workflows, user permissions, email connections, schedules, report filters and sample-data handling. Use version notes to record changes. Keep a rollback plan and test critical paths after publishing. The screenshot shows a trial-expiry notice, so the institution must confirm the licence and ownership arrangement before production use.

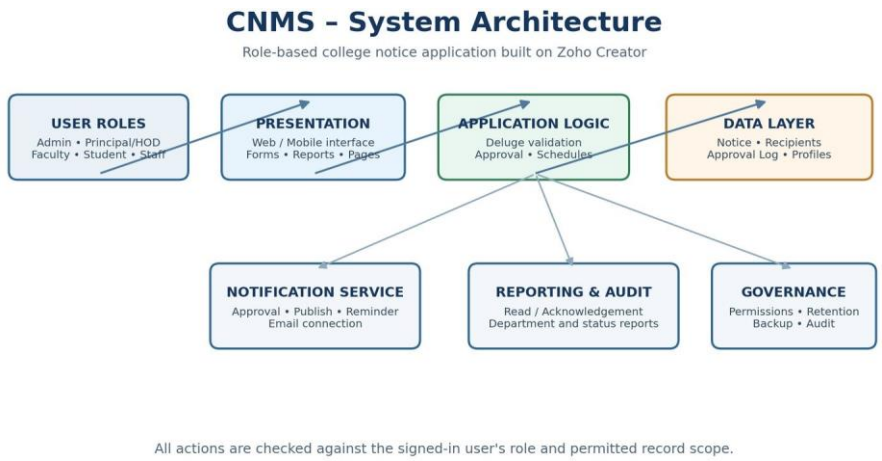


Figure 7.1: [Environments / Publish to Production]

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done (✓)
1	All test cases executed in Development / Stage	[ ]
2	Sample/test data removed or isolated	[ ]
3	Roles, permissions and users configured in Production	[ ]
4	Email connection and sender identity verified	[ ]
5	All schedules enabled and timezone checked	[ ]
6	Backup and restore procedure confirmed	[ ]
7	Mobile interface tested on supported devices	[ ]

7.2 Performance Optimisation

Table 7.2: Performance Optimisation Results

Optimisation Applied	Component	Before	After	Improvement
Filter report records by user/department and status	All Notices / recipient reports	Unfiltered list may load unnecessary records	Criteria-based view	Lower data volume presented
Check for existing notice/user pair before insert	Notify Audience	Repeated schedule could create duplicates	Idempotent recipient creation	Consistent recipient counts
Fetch active profiles using audience criteria	Notify Audience	Repeated broad lookups are inefficient	Single criteria fetch per audience rule	Reduced repeated queries

7.3 Security Features

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Role and record-level permissions	All forms and reports	Unauthorised data access
Authenticated notice and attachment access	Notice detail / file upload	Public exposure of circulars or attachments
Restricted recipient-row access	Notice Recipient	Disclosure of another user's read/acknowledgement status
Authorised email connections	Notification workflows	Credential leakage and unauthorised sending

7.4 Governance and Administration

Governance includes maintaining accurate user profiles, assigning HODs, reviewing role permissions, monitoring failed workflows and bounced email, preserving approval history and defining retention for expired notices. Admin should review inactive accounts and department mappings periodically. Backups and exports must be stored securely. The proposed archival rule moves expired notices to Archived after 90 days, subject to the college's official retention policy.

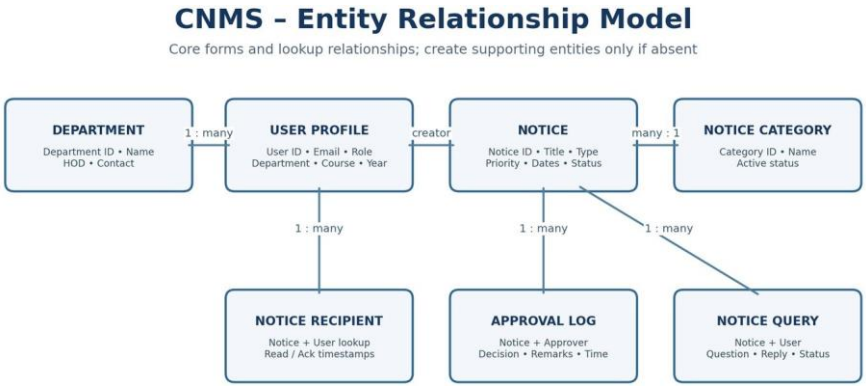


Figure 7.2: [Audit Trail / Administration Settings]

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

A representative scenario begins when a faculty member creates an Exam Schedule Update for all students. The form validates required details and sends the record for HOD approval. The approver checks the dates and content, then approves. When the publication date arrives, the publishing process resolves active student profiles, creates recipient rows and sends notifications. A student opens the notice, which records the first read time, and acknowledges it if required. The faculty member reviews the acknowledgement report.

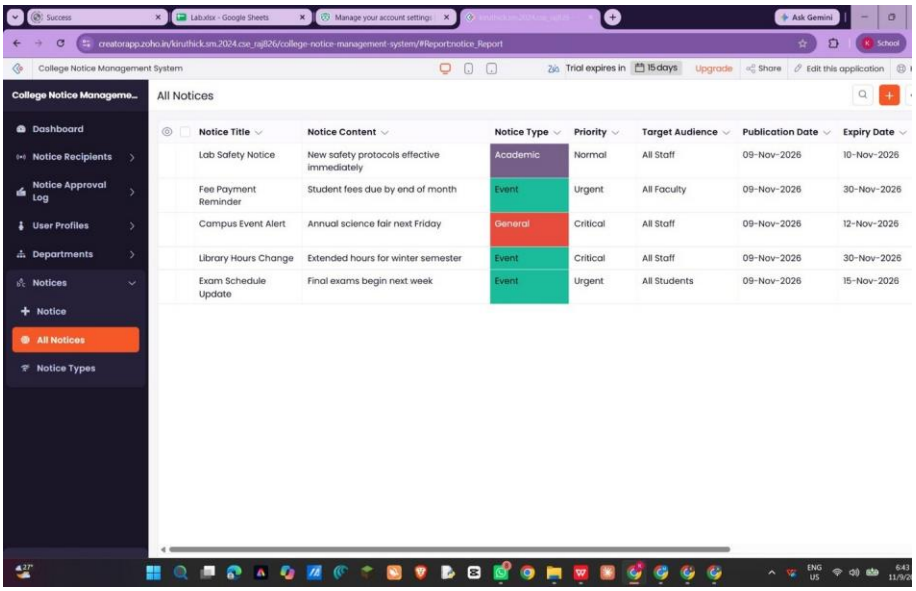


Figure 8.1: All Notices Report – Published Notice Records

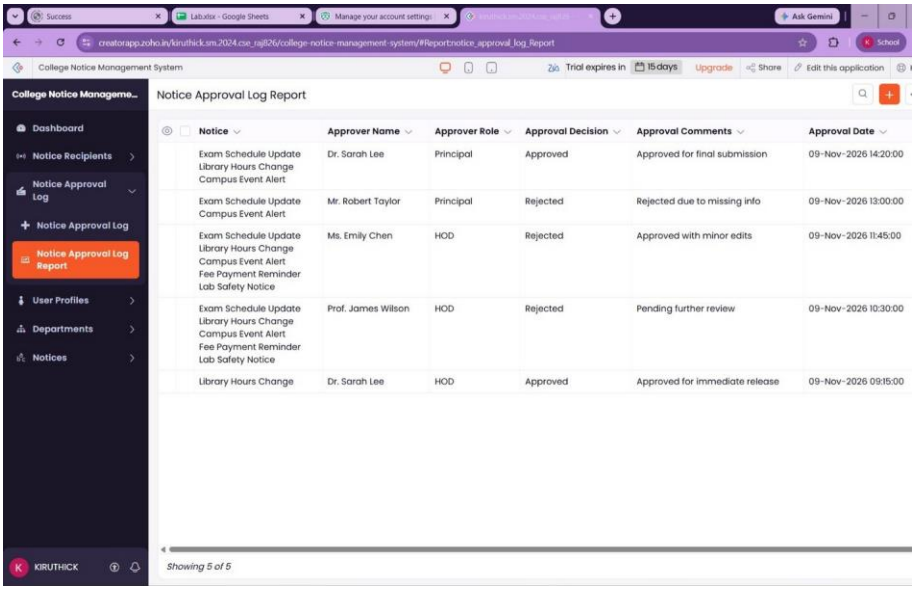


Figure 8.2: Notice Recipient Records and Read/Acknowledgement Tracking

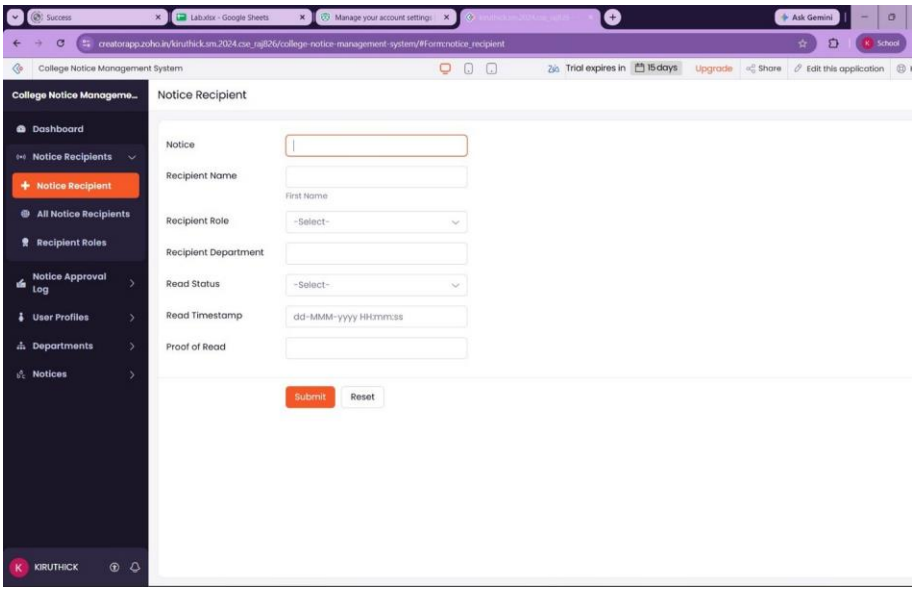


Figure 8.3: Notice Approval Log Report

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Structured data model	Notice, Recipient, Approval Log, User Profile and Department forms	Visible in supplied screenshots; additional fields to verify
Automated notice lifecycle	Deluge workflows and schedules	Specified; live execution evidence required

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Role-based dashboards and reports	Creator pages and report criteria	Dashboard and reports visible; per-role security to verify

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Notice distribution	Multiple channels and manual forwarding	Central repository with targeted recipient records	Single source of current notices
Approval evidence	Informal or separate messages	Approval Log linked to notice	Traceable decision history
Read acknowledgement	Manual follow-up	Recipient-level read and acknowledgement timestamps	Measurable follow-up
Expiry management	Old notices may remain visible	Expiry and archival schedule	Clear lifecycle status

8.4 Limitations of the Low-Code Approach

The practical limitations include dependency on the Creator subscription and quotas, the need to maintain Deluge field link names, constraints on highly customised UI behaviour, and the need to configure integrations and role criteria correctly. Complex audience rules and schedule logic require careful testing. Vendor/platform dependency can be reduced by documenting the data model, workflow logic and export strategy. No specific platform limit or performance benchmark is claimed without measurement.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

The visible Creator configuration is largely no-code: form fields, reports, menus and basic page layouts. Deluge represents the low-code layer for validation, workflow transitions, notifications and audience resolution. A pro-code extension would be justified only for capabilities that cannot be implemented reliably within Creator, such as a specialised external service or custom interface component. The project should remain maintainable by the college's authorised application administrator.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

CNMS establishes a central structure for college notices and connects notice content with approval history, departments, user profiles and recipient activity. The supplied screenshots demonstrate the existing Creator navigation, dashboard, notice list, type grouping, data-entry forms and reports. The project specification defines the additional workflows needed for approval, targeted publishing, read tracking, acknowledgement, queries, reminders and expiry. These automations and permissions must be verified through live tests before the application is treated as production-ready.

9.2 Future Enhancements

- Push notifications and calendar integration for urgent academic events.
- Multilingual notice content and improved search across active and archived notices.
- Integration with the college student information system for synchronised users, courses and academic years.

9.3 Learning Outcomes and Career Relevance

The project develops practical skills in requirements analysis, low-code data modelling, form and report design, Deluge workflow planning, access control, testing and deployment governance. These skills are relevant to low-code application development, workflow automation, business analysis and solutions consulting. The work also demonstrates that a visual builder still requires software engineering discipline, especially around validation, security and test evidence.

REFERENCES

[1] W. Rosam and O. Kalnichevski, *Low-Code Development: A Practical Guide*. New Delhi, India: BPB Publications.

[2] Zoho Corporation, "Zoho Creator Help Documentation." [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: 09 November 2026).

[3] Zoho Corporation, "Deluge Script Reference Guide." [Online]. Available: https://www.zoho.com/deluge/ (Accessed: 09 November 2026).

[4] Zoho Corporation, "Zoho Creator Learning Centre." [Online]. Available: https://www.zoho.com/creator/resources/ (Accessed: 09 November 2026).

[5] O. Matvitskyy, K. Davis and A. Jain, "Magic Quadrant for Enterprise Low-Code Application Platforms," *Gartner Research*, Oct. 2024.

[6] P. M. Gomes and M. A. Brito, "Low-Code Development Platforms: A Descriptive Study," in *Proc. 17th Iberian Conf. Information Systems and Technologies (CISTI)*, Madrid, Spain, 2022, IEEE.

[7] Y. Luo, P. Liang, C. Wang, M. Shahin and J. Zhan, "Characteristics and Challenges of Low-Code Development: The Practitioners' Perspective," in *Proc. ACM/IEEE Int. Symp. Empirical Software Engineering and Measurement (ESEM)*, Bari, Italy, 2021. doi: 10.1145/3475716.3475782.

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	CNMS navigation menu	5.1	Fig. 5.1
II	Forms with multiple field types	Y	Notice, Department, User Profile, Recipient and Approval Log	5.2	Figs. 5.2–5.3
II	Lookup field (relationship between forms)	Y	Notice / User / Department relationships	4.2, 5.2	ER model
II	Subform / parent-child relationship	Y	Recipient and query associations	4.2	ER model
II	Report – List	Y	All Notices / All User Profiles	5.3	Fig. 5.4
II	Report – Grouped / category view	Y	Notice Types	5.3	Fig. 5.4
II	Report – Calendar / timeline	Y	Notice calendar report by publish date	5.3	Added
II	Report – Kanban / status board	Y	Pending approval / lifecycle view	5.3	Added
II	Report filters and sorting	Y	Report columns, filters and sort criteria	5.3	Fig. 5.4
II	Pages and dashboards	Y	Dashboard with KPI cards and charts	5.4	Fig. 5.5
II	Chart / data visualisation	Y	Recipient role and approval decision charts	5.4	Fig. 5.5
II	Deluge form validation	Y	Notice workflow	5.5	DS-01
II	Deluge custom function	Y	Notify_Audience(Notice _ID)	5.5	DS-03
II	Deluge email notification	Y	Approval and publishing workflows	5.5, 5.7	DS-02
II	Workflow – On Validate	Y	Notice form	5.6	DS-01
II	Workflow – On Success / Submit	Y	Notice submission	5.6	DS-02
II	Approval process / custom actions	Y	Notice approval actions	5.6	DS-02
II	Scheduled workflow	Y	Daily publish, reminders and expiry	5.6	DS-05
II	Batch / recurring workflow	Y	Weekly digest and archival	5.6	DS-05
III	Email integration	Y	Authorised Creator email connection	5.7	Email test evidence
III	External API / webhook integration	Y	College SIS integration through REST API	5.7	Added
III	User management and roles	Y	User Profiles and role menu	5.8	Fig. 5.8
III	Record-level permission criteria	Y	Notice, recipient and approval records	5.8	Permission tests
III	Field-level access / visibility	Y	Audience-dependent fields and sensitive data	5.8	Permission tests

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
III	AI / Zia feature	Y	Zia insights on dashboard metrics	5.9	Added
III	Analytics / dashboard metrics	Y	Summary cards and two charts	5.9	Fig. 5.9
V	Pro-code extension	Y	Custom widget and Deluge functions	5.10	Added
V	Mobile app access	Y	Mobile CNMS access	5.11	Mobile test screenshot
V	Customer portal	Y	Student / Staff portal access	5.11	Added
IV	Deployment / production publish	Y	Creator deployment settings	7.1	Deployment evidence
IV	Security / governance controls	Y	Role, record and attachment permissions	7.3–7.4	Security tests
IV	Backup / audit and retention	Y	Admin governance and Approval Log	7.4	Audit evidence

Total features implemented: 32 / 32

APPENDIX B – COURSE OUTCOME MAPPING

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development.	Ch. 1, 2, 4	[ ]	
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5, 6	[ ]	
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs and connections.	Ch. 5.7	[ ]	
CO4	Apply user management, security, deployment and governance controls.	Ch. 4.6, 7	[ ]	
CO5	Evaluate low-code limitations and identify pro-code extension opportunities.	Ch. 8, 9	[ ]	

APPENDIX C – COMPLETE DELUGE SOURCE CODE

C.1 Notice Form – On Validate (DS-01)

```
// Script ID : DS-01
// Form      : Notice        Event : On Add / On Edit > Validate on submit
// Purpose   : Validate mandatory fields, dates, duplicates and attachment type.

if(input.Notice_Title == null || input.Notice_Title.trim() == "")
{
   alert "Notice Title is required.";
   cancel submit;
}
if(input.Notice_Content == null || input.Notice_Content.trim() == "")
{
   alert "Notice Content is required.";
   cancel submit;
}
if(input.Publish_Date == null)
{
   alert "Publication Date is required.";
   cancel submit;
}
// Only Admin may schedule a notice with a past publication date
loginUser = User_Profile[Email == zoho.loginuserid];
if(loginUser.Role != "Admin" && input.Publish_Date < zoho.currentdate)
{
   alert "Publication Date cannot be earlier than today.";
   cancel submit;
}
if(input.Expiry_Date != null && input.Expiry_Date <= input.Publish_Date)
{
   alert "Expiry Date must be after the Publication Date.";
   cancel submit;
}
// Block duplicate notices with the same title, category and date
dupCount = Notice[Notice_Title == input.Notice_Title && Notice_Category == input.Notice_Category &&
Publish_Date == input.Publish_Date && ID != input.ID].count();
if(dupCount > 0)
{
   alert "A notice with the same title, category and date already exists.";
   cancel submit;
}
// Allow only PDF, JPG and PNG attachments
// (the 10 MB size limit is set in the File Upload field properties)
if(input.Attachment != null && input.Attachment != "")
{
   fileName = input.Attachment.toLowerCase();
   if(!(fileName.endsWith(".pdf") || fileName.endsWith(".jpg") || fileName.endsWith(".jpeg") ||
fileName.endsWith(".png")))
   {
      alert "Only PDF, JPG and PNG files can be attached.";
      cancel submit;
   }
}
```

C.2 Function – Notify_Audience (DS-03)

```
// Script ID : DS-03
// Function  : Notify_Audience(int noticeId)
// Purpose   : Resolve matching active profiles, create recipient rows once and send notifications.
```

```
void Notify_Audience(int noticeId)
{
    notice = Notice[ID == noticeId];
    if(notice.Status != "Approved" && notice.Status != "Published")
    {
        return;
    }
    // Resolve the audience from the notice targeting fields
    if(notice.Audience_Type == "Department")
    {
        profiles = User_Profile[Active == true && Department == notice.Target_Department];
    }
    else if(notice.Audience_Type == "Course and Year")
    {
        profiles = User_Profile[Active == true && Course == notice.Target_Course && Year ==
notice.Target_Year];
    }
    else if(notice.Audience_Type == "Role")
    {
        profiles = User_Profile[Active == true && Role == notice.Target_Role];
    }
    else
    {
        profiles = User_Profile[Active == true];
    }
    created = 0;
    for each person in profiles
    {
        // One recipient row per notice and user (no duplicates on rerun)
        if(Notice_Recipient[Notice == noticeId && User == person.ID].count() == 0)
        {
            insert into Notice_Recipient
            [
                Added_User = zoho.loginuser
                Notice = noticeId
                User = person.ID
                Read = false
                Acknowledged = false
            ];
            sendmail
            [
                from :zoho.adminuserid
                to :person.Email
                subject :"[CNMS] " + notice.Priority + " notice: " + notice.Notice_Title
                message :"Dear " + person.Name + ",<br>A new notice has been published for you.<br><br>" +
notice.Notice_Content + "<br><br>Please open CNMS to read" + if(notice.Requires_Acknowledgement ==
true, " and acknowledge", "") + " this notice."
            ];
            created = created + 1;
        }
    }
    notice.Status = "Published";
    notice.Published_On = zoho.currenttime;
    info "Recipients created: " + created;
}
```

C.3 Notice Form – On Success and Approval Actions (DS-02)

```
// Script ID : DS-02
// Form      : Notice        Event : On Add > On Success ; custom actions Approve / Reject
// Purpose   : Route submissions to the HOD, record every decision in the Approval Log.

// ---- On Success (submit) ----
rec = Notice[ID == input.ID];
```

```
creator = User_Profile[Email == zoho.loginuserid];
rec.Created_By = creator.ID;
if(creator.Role == "Admin" || creator.Role == "HOD" || creator.Role == "Principal")
{
   // Auto-approval exception for Admin / HOD / Principal
   rec.Status = "Approved";
   rec.Approver = creator.ID;
}
else
{
   hod = User_Profile[Department == creator.Department && Role == "HOD" && Active == true];
   rec.Status = "Pending Approval";
   rec.Approver = hod.ID;
   sendmail
   [
      from :zoho.adminuserid
      to :hod.Email
      subject :"[CNMS] Approval required: " + rec.Notice_Title
      message :"A notice submitted by " + creator.Name + " is waiting for your approval in CNMS."
   ];
}
insert into Notice_Approval_Log
[
   Added_User = zoho.loginuser
   Notice = rec.ID
   Action_By = creator.ID
   Role = creator.Role
   Decision = rec.Status
   Remarks = "Submitted"
   Action_Time = zoho.currenttime
];

// ---- Custom function used by the Approve / Reject buttons ----
void Decide_Notice(int noticeId, string decision, string remarks)
{
   rec = Notice[ID == noticeId];
   approver = User_Profile[Email == zoho.loginuserid];
   if(decision == "Rejected" && (remarks == null || remarks.trim() == ""))
   {
      info "A rejection reason is required.";
      return;
   }
   rec.Status = decision;
   if(decision == "Rejected")
   {
      rec.Rejection_Reason = remarks;
   }
   insert into Notice_Approval_Log
   [
      Added_User = zoho.loginuser
      Notice = noticeId
      Action_By = approver.ID
      Role = approver.Role
      Decision = decision
      Remarks = remarks
      Action_Time = zoho.currenttime
   ];
   sendmail
   [
      from :zoho.adminuserid
      to :rec.Created_By.Email
      subject :"[CNMS] Notice " + decision + ": " + rec.Notice_Title
      message :"Your notice was " + decision.toLowerCase() + " by " + approver.Name + ". Remarks: " +
ifnull(remarks, "-")
   ];
   // Publish immediately if the publication date has already arrived
```

```
    if(decision == "Approved" && rec.Publish_Date <= zoho.currentdate)
    {
        thisapp.Notify_Audience(noticeId);
    }
}
```

C.4 Notice Recipient – Read and Acknowledge (DS-04)

```
// Script ID : DS-04
// Location  : Notice detail page / Notice Recipient custom actions
// Purpose   : Record the first read time and a separate acknowledgement time.

void Mark_Notice_Read(int noticeId)
{
    me = User_Profile[Email == zoho.loginuserid];
    row = Notice_Recipient[Notice == noticeId && User == me.ID];
    if(row.count() > 0 && row.Read == false)
    {
        row.Read = true;
        row.Read_Time = zoho.currenttime;      // first read only
    }
}

void Acknowledge_Notice(int noticeId)
{
    me = User_Profile[Email == zoho.loginuserid];
    row = Notice_Recipient[Notice == noticeId && User == me.ID];
    if(row.count() == 0)
    {
        info "You are not a recipient of this notice.";
        return;
    }
    if(row.Read == false)
    {
        row.Read = true;
        row.Read_Time = zoho.currenttime;
    }
    if(row.Acknowledged == false)
    {
        row.Acknowledged = true;
        row.Acknowledged_Time = zoho.currenttime;
    }
}
```

C.5 Scheduled Workflows – Publish, Remind, Expire and Digest (DS-05)

```
// Script ID : DS-05
// Schedule  : Daily at 07:00 (publish, reminders, expiry, archive)
//             Weekly on Monday at 08:00 (digest)

// ---- Daily schedule ----
// 1. Publish approved notices whose publication date has arrived
dueNotices = Notice[Status == "Approved" && Publish_Date <= zoho.currentdate];
for each n in dueNotices
{
    thisapp.Notify_Audience(n.ID);
}
// 2. Remind recipients who have not acknowledged required notices
ackNotices = Notice[Status == "Published" && Requires_Acknowledgement == true];
for each n in ackNotices
{
    pending = Notice_Recipient[Notice == n.ID && Acknowledged == false];
    for each r in pending
```

```
    {
        sendmail
        [
            from :zoho.adminuserid
            to :r.User.Email
            subject :"[CNMS] Reminder: please acknowledge " + n.Notice_Title
            message :"This notice requires your acknowledgement. Please open CNMS and acknowledge it."
        ];
    }
}
// 3. Expire notices past their expiry date
expiredNotices = Notice[Status == "Published" && Expiry_Date < zoho.currentdate];
for each n in expiredNotices
{
    n.Status = "Expired";
}
// 4. Archive notices that expired more than 90 days ago
archiveNotices = Notice[Status == "Expired" && Expiry_Date < zoho.currentdate.subDay(90)];
for each n in archiveNotices
{
    n.Status = "Archived";
}

// ---- Weekly digest schedule ----
weekNotices = Notice[Status == "Published" && Publish_Date >= zoho.currentdate.subDay(7)];
if(weekNotices.count() > 0)
{
    summary = "";
    for each n in weekNotices
    {
        total = Notice_Recipient[Notice == n.ID].count();
        acked = Notice_Recipient[Notice == n.ID && Acknowledged == true].count();
        summary = summary + n.Notice_Title + " - acknowledged " + acked + " of " + total + "<br>";
    }
    hods = User_Profile[Role == "HOD" && Active == true];
    for each h in hods
    {
        sendmail
        [
            from :zoho.adminuserid
            to :h.Email
            subject :"[CNMS] Weekly notice digest"
            message :"Notices published in the last 7 days:<br>" + summary
        ];
    }
}
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

1. Open the CNMS application URL provided by the college. 2. Sign in with the assigned account. 3. Use the left navigation menu to open the role-specific dashboard. 4. On mobile, use the supported Zoho Creator app or mobile browser and verify that the notice content and actions fit the screen.

D.2 Instructions for [Role 1]

- 1. Open Notices → Notice and enter title, content, category, priority, audience and schedule. 2. Attach an allowed file if needed and submit for approval. 3. Monitor your notice status and read/acknowledgement report.
- 4. If rejected, review the rejection reason, edit the notice and resubmit. 5. Use the query/report views to follow questions and recipient responses where those features are enabled.

D.3 Instructions for [Role 2]

- 1. Open the pending approval view and inspect notice details, audience, priority, dates and attachment. 2. Select Approve or Reject; enter a reason when rejecting. 3. Verify that the decision appears in the Approval Log.
- Student/Staff: open published notices targeted to your role, use search and filters, read the notice, acknowledge when required and submit a query if enabled.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	NARENDARAN K	Register No.	240701624
Project Title	College Notice Management System	Date	11/11/2026

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clear CNMS problem, objectives, stakeholders, FR/NFR and use cases	10	
2	System Design	Architecture, data model, form specifications, workflows and permission matrix	10	
3	Forms, Reports & Pages	Notice forms, lookups, reports, dashboard and usability	15	
4	Deluge and Automation	Validation, approval, audience resolution, schedules and notification logic	15	
5	Integration and Security	Email connection, role/record permissions and data protection	10	
6	Testing and Debugging	Test coverage, negative cases, defect log and UAT evidence	10	
7	Deployment and Governance	Deployment checklist, performance, backup and audit	10	
8	Documentation and Demonstration	Report quality, screenshots, live workflow demonstration	10	
9	Viva-Voce	Conceptual understanding and explanation of design decisions	10	
	TOTAL		100	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and supported by clear test evidence; student explains design decisions.
Good	70–89	Feature implemented correctly with minor gaps; explanation and evidence are adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; testing evidence is limited.
Needs Improvement	Below 50	Feature is missing, unreliable or not explained; corrective work is required.

Signature of Internal Examiner
Name: _____

Signature of External Examiner
Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	NARENDARAN K	Register No.	240701624
Department / Section	CSE	Date of Viva	11/11/2026
Application URL Verified	Yes / No	Demo Duration	_____min

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1	Explain the CNMS notice lifecycle and status transitions.	II		
2	How does the system decide which users receive a notice?	II		
3	Explain the difference between Read and Acknowledged.	II		
4	How does the Approval Log support accountability?	II		
5	How would you prevent duplicate recipient records?	II		
6	How are role and department permissions enforced?	III		
7	Which schedules are required and what do they do?	II		
8	What is the role of Deluge in CNMS?	II		

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	**100**	

Examiners' Remarks:

_____

_____

INTERNAL EXAMINER
Name: _____

EXTERNAL EXAMINER
Name: _____